A Report on the Implementation and Performance of a Work Stealing Loop Scheduling Algorithm in OpenMP

Exam No. B024703

November 29, 2016

1 Introduction

This report details experiments undertaken to implement a work stealing algorithm in OpenMP. The algorithm, known as affinity scheduling, assigns work by giving blocks of work to each thread but allowing threads that have completed their block to take work from another.

The subject of this experiment was a code that contained two loops, with iterations spread evenly across threads. The author then refactored this code to use an affinity scheduling algorithm. Suboptimal parallelisation of a loop can occur when a thread finishes work that it has been assigned and must remain idle until others have completed. Affinity scheduling is a strategy for minimising this idle time. As part of the experiment, this scheduling technique was compared with the scheduling options that are defined in the OpenMP standard. The work sharing strategies defined by the standard that achieved the best results on this code were determined by a previous experiment. It is these best schedules that are used for comparison in this report.

2 Methods

2.1 Compiling and Running the Original Code

As a precaution, to check the correctness of any subsequent version, the original code was compiled in its original form. This form could be thought of as an explicitly implemented version of the static loop scheduling option in OpenMP, with the iterations of the loops evenly divided between threads. This compilation was done using the Intel C Compiler with the `-O3` option specified at compile time for maximum serial optimisation. Beyond using this to verify the correctness of any future solution and that performance was credible, analysis of the performance of this code falls outside the scope of this report.

2.2 The Algorithm

Each thread is initially assigned an equal amount of contiguous iterations. Where the number of iterations does not evenly divide between threads, each thread is given $ceil(n/p)$, where n is the total number of iterations and p is the total number of threads, apart from the final thread, which gets a smaller block such that the total number of iterations divided between threads is correct.

Every thread then executes blocks of iterations whose size is $ceil(1/p)$ of the remaining iterations its local set. This continues until there are no more iterations left in the set of iterations it was initially assigned.

When a thread reaches this point it then scans the other threads to see which has the most remaining of its initial set and executes $ceil(1/p)$ of these.

While a thread has no more of its own initial set left, it continues perform this loop of scanning to see which thread has the most iterations remain and executing a block from this until there are no more iterations of the loop left to compute on any thread's local set.

This algorithm has good theoretical potential. It allows each thread to maintain a contiguous block, which is beneficial for reasons of cache coherency, while preventing threads from idling by allowing them to steal work once they have finished this contiguous block.

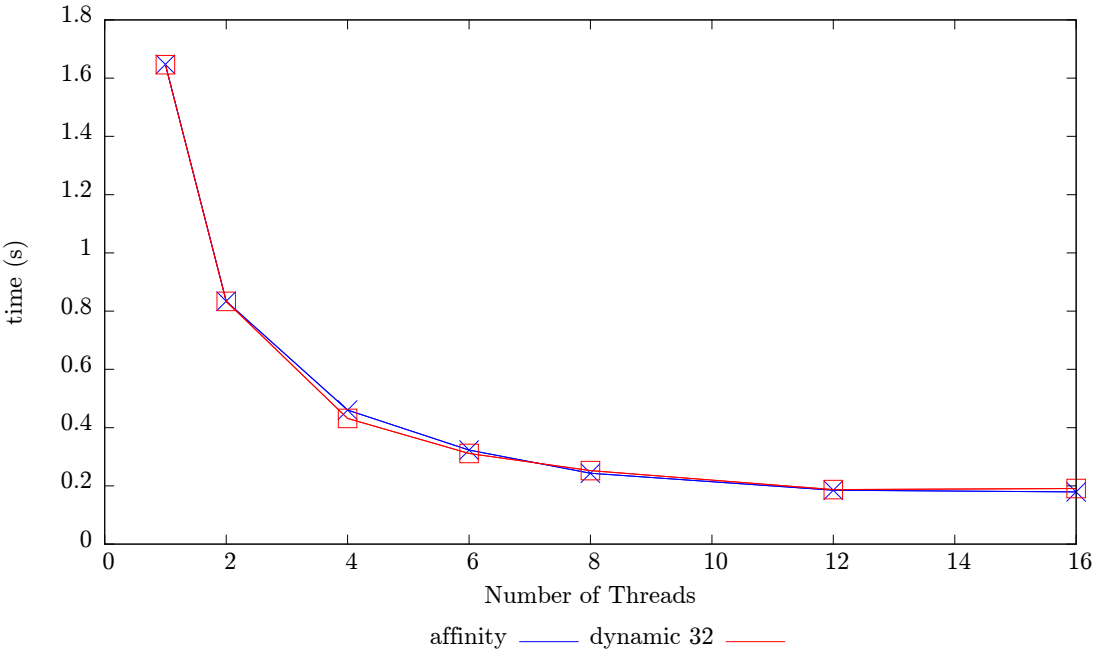

Figure 1: Graph showing execution time for loop 1 against number of threads for the affinity and dynamic, 32 schedules. Note the performance is extremely similar.

2.3 Implementation

This algorithm is non-trivial to implement as it involves a great degree of synchronisation between threads, as well as requiring that each thread be aware of the progress of each other.

The awareness was maintained with the use of a shared variable which recorded the state of the algorithm. This was an array, `remaining[]` containing the amount of iterations that had yet to be computed from each threads initial set. Each time a thread executed a block of size n from a set i of iterations it would decrement `remaining[i]` by n.

In terms of synchronisation it was important not only that no two threads attempt to modify this array simultaneously, but that no thread attempts to modify it while another is scanning to see which thread has the most iterations remaining. To prevent these clashes a critical region was used. According to the OpenMP standard, only one thread may enter this region at a time. This, however, can have negative effects on performance, particularly in highly parallel cases, as no thread can can be assigned work while another is.

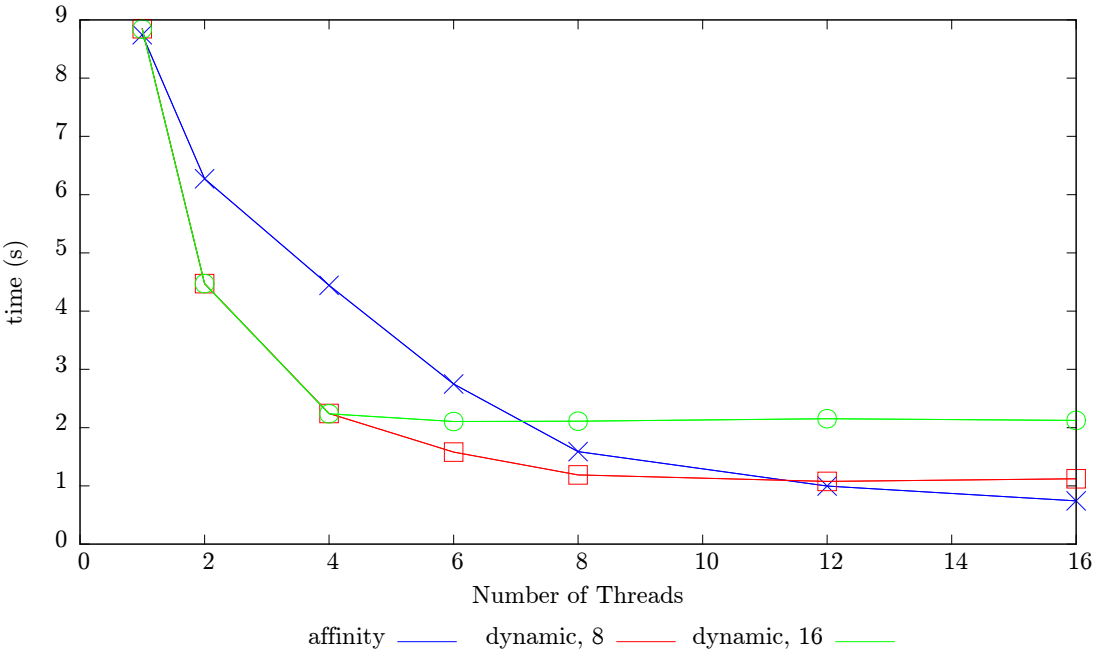

Figure 2: Graph showing execution time for loop 2 against number of threads for the affinity, dynamic, 8 and dynamic, 16 schedules.

To go into more technical detail, the work assignment was lifted into a function. This function takes as input an array of the amount of remaining iterations, an array of the end index of each local set - `end_index` (this could theoretically be derived each time but this would be less performant), the thread id of the thread requesting work and the total number of threads. It gives as an output values `hi` and `lo`, between which is the contiguous block of iterations that the thread should execute. The function also has a return value which it sets to true when it determines all work is complete.

The function works by first checking `remaining[my_id]` to see if there is any more work in the calling threads local set. If there is, then it assigns a local variable `steal_id` to be equal to the threads own id. Otherwise, the thread scans the `remaining` array to find the highest value, and sets the `steal_id` variable to the index that contains this highest value. Where all elements of the `remaining` variable are zero, the loop is complete and the function will return 1. In other cases the function instructs the thread to compute `ceil(remaining[steal_id]/p)` iterations from the work associated with the thread `steal_id`. It does this by setting the `lo` variable to `end_index[steal_id]-remaining[steal_index]` and the `hi` variable to this plus `ceil(remaining[steal_index]/p)`.

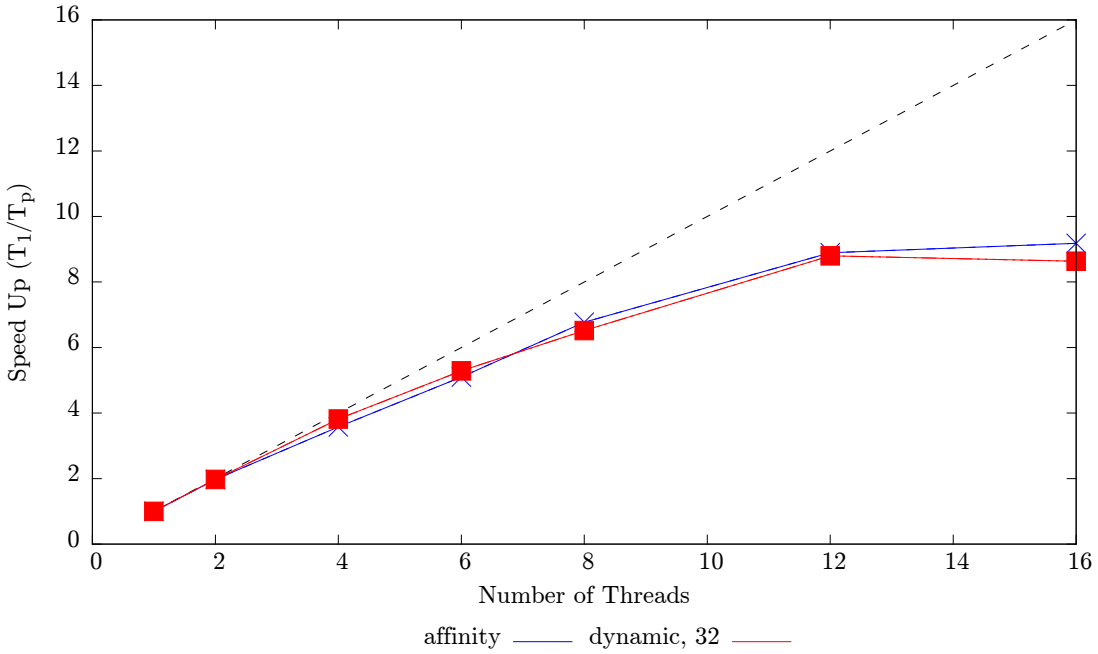

Figure 3: Speed Up of loop 1 for the dynamic 32 schedule. A theoretical ideal speed up is shown as a black dotted line.

The function as implemented is included as appendix B

3 Results

3.1 Correctness

For almost all HPC applications correctness is the primary concern. The code provided included a check sum that could be used to verify the correctness of the affinity results. In all cases this sum yielded the same result as the code provided. We can therefore disregard correctness as a factor when comparing the different schedules.

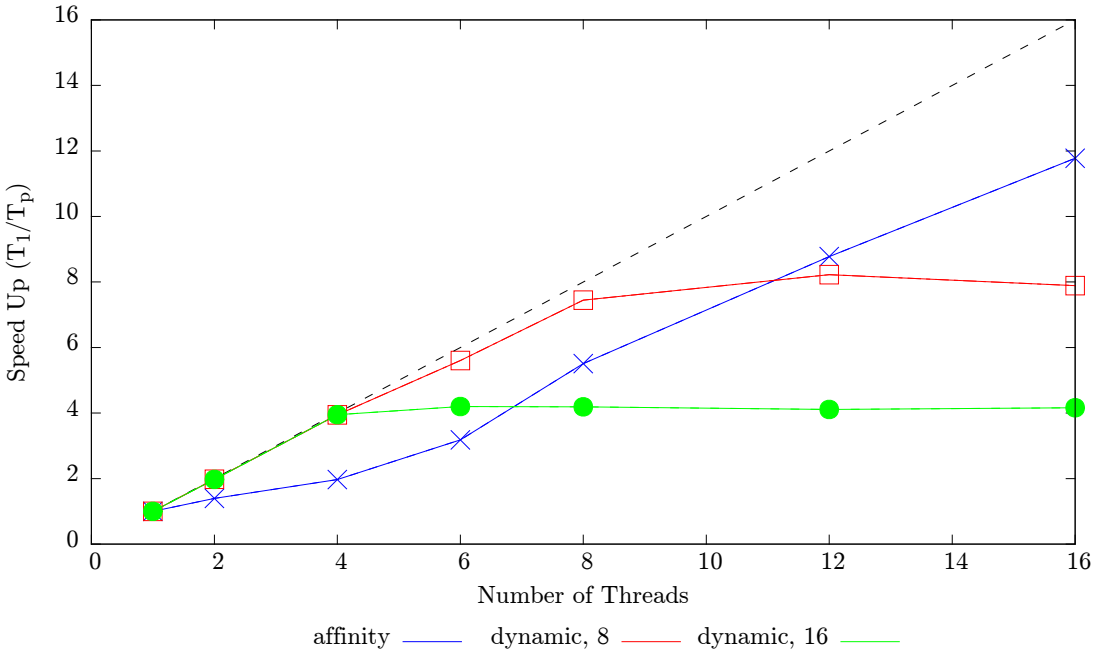

Figure 4: Speed Up of loop 2 for the affinity schedule. A theoretical ideal speed up is shown as a black dotted line.

3.2 Comparative Execution Times of Schedules

The first results gathered were execution times for each loop with both the affinity scheduling and the best loop schedule that comes by default with any OpenMP implementation. These best schedules were determined and discussed in an earlier report. The best schedule for loop 1 was `dynamic, 32` while for loop 2 the results for `dynamic, 8` and `dynamic, 16` were similarly optimal enough that they are both used here. The results for the execution time of loops 1 and 2 are shown in figures 1 and 2 respectively.

3.2.1 Loop 1

For loop 1 we see that performance yielded from the loops running with affinity scheduling is almost identical to that yielded from the `dynamic, 32` schedule. Such similar performance is explainable, as the scheduling techniques are quite similar in theory. As explained in [1], the `dynamic, 32` schedule has one work queue that is shared between all threads. Each thread claims a contiguous block of 32 iterations from this queue and executes it before going back to the queue

to claim another. As the loop in question is natural fairly well load balanced, or each iteration takes a similar amount of time to execute, the main differences between performance from different schedules on this loop come from the overhead of assigning work. Although this work assignation process happens very differently on the two schedules trialled on this loop, the end result seems to be similar. It is worth noting that the affinity schedule does not gain significant performance improvements from cache coherency in this case.

It is likely the same issues become limiting factors for horizontal scalability once the number of processes reaches about 12. This is that threads have to wait to be assigned work for reasons of synchronisation for both these strategies.

3.2.2 Loop 2

Loop 2 shows some more interesting differences between scalability of the two strategies. The `dynamic, 8` and `dynamic, 16` perform similarly to each other until the number of threads reaches 4. At this point no further speed up is shown where the block size is 16. This is because this loop is inherently badly load balanced, meaning that the slowest block of size 16 becomes the dominating factor of the computation time when the `dynamic, 16` schedule is selected. Similarly, the `dynamic, 8` schedule's performance is limited by the time taken for the slowest block of 8 to execute. As expected this approximately half the time taken for the slowest 16 iterations to execute, meaning that the shortest time achieved using this schedule is approximately half that of the time achieved with the larger block size.

For the affinity scheduling the initial block size can be thought of as proportional to $1/p^2$, where p is the total number of threads. This is because each thread starts off with a local set of $s = 1/p$ iterations, before iterating through a block of s/p. This explains the affinity schedule's relatively poor performance where p is smaller, as this corresponds to a large effective block size. The total number of iterations executed was $T = 729$. Therefore, if the execution time of the slowest block were a completely dominating factor, we would expect the performance of the affinity schedule to eclipse that of `dynamic, 16` where $T/p^2 = 16$, or $p = 6.75$ and that of `dynamic, 8` where $T/p^2 = 16$, or $p = 9.5$. Interpolating from figure 2 we can see these calculated figures are broadly correct. The fact that it is more like $p = 11$ before the performance of affinity and `dynamic, 16` could be explained by the greater deal of synchronisation required for the affinity schedule. This is because the blocks become smaller as the process goes on, meaning more time is spent fetching work - requiring threads to enter or wait to enter a critical region.

4 conclusion

In conclusion we see that affinity scheduling is a broadly effective strategy. For at least some values of p this schedule matched or bettered the performance of the best option from the OpenMP standard. This is no small feat considering the OpenMP implementation used has the full weight of the mighty Intel Corporation behind it, and the schedules themselves may or may not have been implemented using a lower level, faster framework than OpenMP itself. This suggests that the implementation of this schedule by the author was of high quality.

However, there were no demonstrable speed ups shown to be from cache coherency effects, which is one of the justifications for this technique. Indeed, it seems likely that in the loop 2 case that affinity scheduling only provided higher horizontal scalability because of a lower effective block size. It is therefore likely that similar or better performance could be achieved where p is larger simply by using the dynamic schedule with a smaller block size. This would be a worthwhile future experiment, but goes beyond the scope of this report.

References

[1] Intel Corp. *OpenMP Loop Scheduling* `https://software.intel.com/en-us/articles/openmp-loop-scheduling` Accessed: 26-10-2016

A Full Results

Table 1: Loop 1

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
dynamic,32	1	1.64	0.00
dynamic,32	2	0.83	0.00
dynamic,32	4	0.43	0.01
dynamic,32	6	0.30	0.01
dynamic,32	8	0.23	0.01
dynamic,32	12	0.16	0.01
dynamic,32	16	0.13	0.01
affinity	1	1.65	0.00
affinity	2	0.83	0.00
affinity	4	0.46	0.00
affinity	6	0.32	0.03
affinity	8	0.24	0.01
affinity	12	0.19	0.01
affinity	16	0.18	0.01

Table 2: Loop 2

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
dynamic,8	1	8.84	0.01
dynamic,8	2	4.47	0.02
dynamic,8	4	2.24	0.01
dynamic,8	6	1.57	0.00
dynamic,8	8	1.18	0.00
dynamic,8	12	1.07	0.01
dynamic,8	16	1.12	0.10
dynamic,16	1	8.84	0.00
dynamic,16	2	4.47	0.02
dynamic,16	4	2.23	0.01
dynamic,16	6	2.10	0.01
dynamic,16	8	2.10	0.01
dynamic,16	12	2.15	0.09
dynamic,16	16	2.12	0.04
affinity	1	1.65	0.00
affinity	2	0.83	0.00
affinity	4	0.46	0.01
affinity	6	0.32	0.03
affinity	8	0.24	0.01
affinity	12	0.19	0.01
affinity	16	0.18	0.01

B Code Snippet

```
int get_work(int *remaining, int *end_index, int myid, int nthreads, int *lo,
             int *hi)
{
  int i, chunk_size, steal_index, temp_max;
  temp_max = 1;
  // if a thread has no more work it should steal work.
  if (remaining[myid] == 0)
  {
    // use loop to find thread with most work remaining
    temp_max = 0;
    int i;
    for (i = 0; i < nthreads - 1; i++)
    {
      if (temp_max < remaining[i])
      {
        temp_max = remaining[i];
        steal_index = i;
      }
    }
  }
  else
  {
    steal_index = myid;
  }

  if (temp_max != 0)
  {
    chunk_size = (int)ceil((double)remaining[steal_index] / (double)nthreads);
    *lo = end_index[steal_index] - remaining[steal_index];
    *hi = *lo + chunk_size;
    remaining[steal_index] -= chunk_size;
    return 0;
  }
  else
  {
    return 1;
  }
}
```